

Noise-Tolerant Plaintext-Checking Oracle Attacks

A Soft-Analytic Approach Applied to ML-KEM

Julius Hermelink¹, Erik Mårtensson^{2,3}, and Maggie Tran²

¹ Max Planck Institute for Security and Privacy, Bochum, Germany,
julius.hermelink@mpi-sp.org

² Lund University, Lund, Sweden, {erik.martensson,maggie.tran}@eit.lth.se

³ Advenica AB, Malmö, Sweden

Abstract. Plaintext-checking (PC) oracle attacks are among the most prominent types of attacks against NIST’s recently standardized ML-KEM. Previous works have drastically reduced the number of queries needed to recover the secret key. Although this number is now close to the information-theoretic bound, current attacks have not yet been adapted to settings with increased noise and highly imperfect oracles. In attacks targeting real-world protected implementations, noisy leakage may lead to oracles that only provide a small advantage over guessing.

This work shows how to efficiently exploit imperfect oracles arising from highly noisy side channels. We present several soft-analytic techniques that enable highly noise-tolerant parallel PC-oracle attacks. These techniques allow for successful attacks in relatively few traces from information that gives just a slight advantage over guessing. Additionally, we extend the generic framework for side-channel information from Eurocrypt 2025 and thereby relate our techniques to previous work.

We then discuss several oracle instantiations that are based on the noisy Hamming weight model. These oracles rely on widely accepted assumptions, are easy to simulate, and allow for fair comparisons between different attacks. Furthermore, they allow countermeasures to be taken into account, and we evaluate the impact of masking. Our evaluations in these and previous models show that PC-oracle attacks are noise-tolerant on an entirely different scale compared to previous work. These improvements are of an algorithmic nature and orthogonal to the fact that the Fujisaki-Okamoto transform in ML-KEM offers a large attack surface. Finally, we discuss the implications of our findings for protected ML-KEM implementations.

1 Introduction

The National Institute of Standards and Technology (NIST) standardization process for post-quantum cryptography is still ongoing, and the first schemes have been standardized [40]. This includes the lattice-based schemes Module-Lattice-based Key-Encapsulation Mechanism (ML-KEM) [39] and Module-Lattice-based Digital Signature Algorithm (ML-DSA) [38], both relying on a variant of the Learning with Errors (LWE) problem [51] called Module Learning with Errors

(MLWE) [32]. Real-world messaging applications such as Signal and iMessage already employ ML-KEM [31,11] and Cloudflare reported that 13.0% of the connections relied on ML-KEM already in November 2024 [2]. In the area of side-channel attacks on LWE-based schemes, one of the more prominent and investigated types of attacks is the so-called Plaintext-Checking (PC)-oracle attacks (see, e.g., [49]). These attacks exploit that ML-KEM relies on a Fujisaki-Okamoto (FO) transform [13,14] to derive a Key Encapsulation Mechanism (KEM) from a Public-Key Encryption (PKE), thereby achieving IND-CCAs (Indistinguishability under Chosen-Ciphertext Attacks). In practice, this means that the KEM version not only decrypts a ciphertext, but also re-encrypts it. The re-encrypted ciphertext is compared with the submitted one, and the ciphertext is rejected if they do not match, preventing Chosen-Ciphertext Attacks (CCAs). However, side channels might leak information during the re-encryption or the comparison, thereby allowing for CCAs. PC-oracle attacks exploit leakage on the decrypted message, which enables key mismatch attacks [12,50] on the underlying PKE from which the KEM is derived using the FO-transform.

Although generic in theory, PC-oracles have mainly been realized through power or electro-magnetic radiation side channels. Most works have focused on reducing the number of queries in key mismatch attacks, which corresponds to the number of traces required in a Side-Channel Attack (SCA). Key mismatch attacks against lattice-based schemes were first introduced by Fluhrer in [12] and later applied to ML-KEM⁴ in [46], requiring 1855 queries for Kyber768. Subsequent work in [47] reduced this to 1776 queries (as clarified in [18] for Kyber1024), and [18] further lowered the requirement to 1588 queries. The authors of [19] improved the query complexity, explained how the balance between the cost of queries and post-processing in a parallel mismatch attack can be improved, and thereby showed that giving the adversary as few as two (perfect) mismatch queries is enough to break the security level of ML-KEM.

Several works have also investigated practical attacks: Binary PC-oracle attacks, i.e., instantiations of key mismatch attacks using a side channel on ML-KEM and other lattice-based schemes were studied in [50]. More recent work [48] showed that multiple coefficients can be targeted in parallel if the adversary can distinguish more than two messages (a concept also proposed in [52]), effectively reducing the number of required queries by a factor of N , compared to [50], when targeting N positions in parallel.

The number of traces is a commonly used metric in SCAs: For masking countermeasures [5], we can expect the leaked information to decrease exponentially in the number of shares *if sufficient noise is present* [9]. Therefore, the number of traces an attack requires is a relevant metric, as it is expected to grow exponentially with the number of shares in many attacks. However, noise conditions are another important factor to consider. Without noise, it is unsurprising that higher-order attacks succeed, as (Boolean) masking may provide little or no reduction on leaked information (see, e.g., [35]). Boolean masking (in contrast to arithmetic masking) offers particularly little benefit in low noise

⁴ We do not differentiate between Kyber and ML-KEM.

conditions [35,23]. Moreover, not taking noise conditions into consideration when designing countermeasures can lead to devastating attacks [24,27].

In PC-oracle attacks, adversaries may target a comparably long-running computation. They are only required to distinguish 2^N messages, where N is the number of targeted positions in parallel and can be set to 1. Therefore, attacks under low-noise conditions or targeting unprotected implementations achieve high accuracy, especially when evaluations are performed on platforms with relatively low noise rather than real-world devices. As a result, handling higher noise levels or imperfect oracles has not been necessary for successful attacks, even on physical devices and masked implementations (see, e.g., [56,55,48]). However, real-world protected implementations are expected to exhibit higher noise levels and employ various countermeasures, particularly given that prior work has highlighted the threat posed by PC-oracle attacks.

While some works have considered imperfect oracles, the area is relatively underdeveloped: Dong and Guo [8] recently showed how to deal with soft information in PC-oracle attacks against HQC [1] using an enhanced information set decoding algorithm. However, these techniques do not translate to PC-oracle attacks on ML-KEM, and the oracle itself is assumed to give hard information. For ML-KEM512, the approach of [53] has an error rate of 0.25 coefficients per key for an oracle accuracy of $\alpha = 0.9$. At the same accuracy level, $\alpha = 0.9$ (considered to be a high-noise setting), an error rate of 0.646 coefficients per ML-KEM1024 key for targeting $N = 8$ positions is reported by [33] while already relying on majority voting and requiring 2041.6 queries. For comparison, our binary PC-oracles with $\alpha = 0.51$ can be targeted with an error rate of less than 0.001 per subkey; for $N \in \{4, 8\}$ and $\alpha \in \{0.9, 0.95\}$, we observe no failure in 10000 experiments. As a result we require just around 700 queries at $N = 8$ and $\alpha = 0.9$, and around 1000 traces at $\alpha = 0.7$ compared to the around 7000 queries reported in [33]. Further, attacks in [33] require significantly more queries to achieve relatively low error rates, particularly as α decreases (see Table 2). Moreover, these works assume a “tournament-based” oracle. Such an oracle is instantiated by comparing how well, e.g., a template fits two messages and choosing the more likely one until no more potential messages are left – just as in a tournament; the remaining message is assumed to be the correct one. This approach may result in a considerable loss of information (see Figure 6): the probability for the returned message to be correct, an information often obtained in common attacks, cannot be considered in this abstraction.

In summary, targeting the FO-transform allows for distinguishing messages with high accuracy – even in masked implementations (see, e.g., [56]). Previous attacks assume similar accuracies and focus on low-noise settings, whereas real-world implementations aim to reduce the attack surface using masking and hiding countermeasures under sufficient noise. Designing effective countermeasures requires understanding how much of an advantage in distinguishing messages translates into the number of required traces for a successful attack. Moreover, higher-order masking is costly to implement, and the assumption that these attacks quickly deteriorate for imperfect oracles may lead to insufficient

countermeasures. Decryption-failure attacks [17] have been shown to be highly noise-tolerant [25,22]⁵, yet handling highly imperfect and noisy oracles in PC-oracle attacks remains an open question. This question is orthogonal to the fact that the FO-transform offers a large attack surface.

Our contribution. In this work, we present an information-theoretic approach to PC-oracle attacks. We define a new abstraction for oracles that allows for improved modeling. Furthermore, we employ soft-analytic [57] techniques to deal with noisy and imperfect PC-oracles. To our knowledge, this is the first soft-analytic approach to PC-oracle attacks⁶.

First, we present several new attacks that range from being computationally expensive but optimally exploiting all information to being very efficient but losing some information. The optimal attack keeps track of the joint probability distribution of all targeted key coefficients conditioned on the information given by the oracle. The most efficient version only keeps track of an approximation of the marginals of the joint probability distribution. The former is computationally expensive – but easily carried out on a laptop for ≤ 5 positions in parallel in seconds – while the latter uses the information less efficiently. As a trade-off, we provide a Belief Propagation (BP) instantiation that (better) approximates the marginals of the joint probability distribution. For the optimal attack, we discuss different ways to choose oracle queries – again providing a trade-off between loss in computational and information-theoretic efficacy. For imperfect oracles, our attacks require fewer queries compared to previous works.

Secondly, we analyze the shortcomings of the generic framework for side information presented in [26]. We note that the definition of distribution hints always assumes independence and does not allow for hints that are correlated. To address this limitation, we introduce *joint distribution hints*, extending the solver of [26] to our definition with an optimized implementation⁷.

Thirdly, we discuss several oracle instantiations and define two new models for evaluating PC-oracle attacks. These oracles allow for a unified evaluation of attacks while being rooted in a realistic, long-standing assumption, namely the noisy Hamming weight (HW) model. Our oracles assume that the adversary is able to obtain the noisy HW of k uniformly random intermediate values during the re-encryption; this can be done by, e.g., targeting the last layer of the Number Theoretic Transform (NTT). Each value in turn corresponds to $t + 1$ shares that mask a secret, or the value itself in an unmasked implementation. We provide an information-theoretic analysis of the previously used tournament-based oracle and discuss such evaluations for the noisy HW-based models.

Finally, we evaluate our attacks in the presented oracle models. Overall, the optimal and the BP-based attacks have (almost) no failures when using a sufficiently strict abort criterion. Our criterion, the entropy of the subkey being less than 0.001, resulted in a single failure in all settings in the preliminary evaluations

⁵ The fault attack obtains the same information as decryption-failure attacks.

⁶ Apart from instantiating an oracle using BP.

⁷ For our evaluations we provide a more intuitive Python implementation.

and none in the final evaluations. Stricter abort criteria allow further reducing the failure rate. Moreover, we show that even naïve soft-analytic approaches achieve considerable improvements for imperfect oracles. We target an abstract tournament-based oracle and simulate leakage for first-, second-, and third-order leakage in masked implementations. In all settings, our attacks achieve failure rates close to zero under drastically increased noise compared to previous attacks.

Overview of results. For binary PC-oracle attacks, we show that an adversary may successfully use a tournament oracle with $\alpha = 0.51$ at less than one error per 10000 samples⁸ without repeating queries. To fully recover an ML-KEM1024 when targeting $N = 8$ positions we require 1529.6 queries at $\alpha = 0.5$, 706.56 at $\alpha = 0.9$, and 944.64 at $\alpha = 0.7$ ([33] requires over 7000 queries at $\alpha = 0.7$). The noisy HW-based oracles depend on the number of values we target in the NTT during the re-encryption and the standard deviation σ of the noise. These values are targeted individually, and we exploit no relations between them. For 100 targeted values, we require 238 traces at $\sigma = 3$ and 1835 at $\sigma = 10$. Targeting the last layer of the NTT in a second-order attack requires just 742 traces for full key-recovery at $\sigma = 1.0$. Furthermore, we show that even with just a *single intermediate* for which both shares leak, we may on average recover the secret key with less than 270000 traces at $\sigma = 1.0$ in a first-order masked implementation.

Practical implications. These attacks are known to not require a clone device (see, e.g., [48]). Moreover, the attack surface is large and allows for targeting many values that hold algebraic relations, e.g., during the NTT or Keccak. These have previously been exploited in different contexts [45,43,29,20]. Furthermore, the work of [56] achieves very high success rates for a masked implementation. Our improvements in this work are orthogonal to the large attack surface. We demonstrate that even with a tiny attack surface, these attacks are highly noise tolerant and potentially devastating. Thereby, we show that these attacks present an even greater threat than previously believed.

Open source. Our simulations are written in Python and rely on PQCclean [30]. The generic BP-based solver described in Section 3.3 is written in Rust and extends the implementation of [26]. Note that we do not provide an implementation for the greedy solver we additionally describe in Section 3.3, and we also provide no evaluations in more general settings for the BP-based solver; we leave this for future work. Our implementations are open-source and can be found under https://github.com/juliusjh/sasca_imperfect_pc_oracles.

2 Preliminaries

ML-KEM has been standardized by NIST as the first post-quantum KEM, and PC-oracle attacks are among the most notable side-channel attacks against ML-KEM. We show that soft-analytic techniques enable far more noise-resistant

⁸ This means very little information; $\alpha = 0.5$ corresponds to no information for $N = 1$.

PC-oracle attacks and require far fewer traces, compared to previous works. We first introduce notation, then give an overview of ML-KEM [39], PC-oracle attacks, and a Soft Analytical Side-Channel Attack (SASCA) [57].

2.1 Notation

Vectors and matrices are denoted in bold, e.g., $\mathbf{a}$ and matrices are capitalized. For values, e.g., $m \in \mathbb{Z}$, the notation m_i denotes the i -th bit. The secret, usually an N -dimensional subkey of the ML-KEM key, is denoted by $\mathbf{s}$. It is assumed to be drawn from a centered binomial distribution $\mathbf{B}_\eta$ with parameter η , i.e., the distribution arising from sampling 2η bits and subtracting η from the number of sampled ones. We write PC-oracles as $\mathcal{O}$, which can be queried for a vector $\mathbf{a}$, representing smaller-equal inequalities in the secret. The oracle responds with a message m that can be interpreted as $\mathbf{m} = (m_0, \dots, m_{N-1})$, where 1 corresponds to “true”. By $[K]$ we mean $\{0, \dots, K-1\}$. We often interpret polynomials in $R_q = \mathbb{F}_q[x]/(x^n - 1)$ as vectors when the multiplicative structure is irrelevant.

2.2 ML-KEM

ML-KEM is the first Post-Quantum (PQ) KEM standardized by NIST. Its security is based on the MLWE problem, a variant of the LWE problem. LWE asks to recover $\mathbf{s} \in \mathbb{F}_q^k$ from samples $(\mathbf{a}, \mathbf{a}\mathbf{s} + e) \in \mathbb{F}_q^k \times \mathbb{F}_q$ where $\mathbf{a}$ is uniformly random, and e follows a “small” error distribution. In practice, both $\mathbf{s}$ and e are often sampled from the same distribution and the number of samples usually equals the dimension of $\mathbf{s}$. The problem can be written in matrix form as recovering $\mathbf{s}$ given $\mathbf{A}\mathbf{s} + \mathbf{e}$ and $\mathbf{A}$. In the MLWE problem, the coefficients of $\mathbf{s}$, $\mathbf{a}$, and $\mathbf{e}$ are chosen from R_q^k where $R_q = \mathbb{F}_q[x]/(x^n - 1)$, enabling fast multiplication using an NTT. For $k = 1$, this gives the Ring Learning with Errors (RLWE) problem; for $n = 1$, we recover the LWE problem. Solving MLWE – for parameter sets where it is feasible – is usually done by seeing it as an LWE problem.

In ML-KEM, $\mathbf{s}, \mathbf{e}$ are sampled from $\mathbf{B}_\eta$, with η set to 3, 2 and 2 for security levels 512, 768 and 1024, respectively. The value of n is set to 256 and $q = 3329$. The value of k is 2, 3, or 4, resulting in $\mathbf{e}, \mathbf{s}$ being of dimension 512, 768, 1024, respectively, corresponding to the security levels.

ML-KEM first defines an IND-CPA-secure PKE, then uses an FO transform to obtain an IND-CCA2-secure KEM. Intuitively, the KEM deterministically encrypts a randomly sampled message $\mathbf{m}$, then, instead of just decrypting the ciphertext, the other party decrypts and then re-encrypts the message. This yields a re-computed ciphertext which is then compared against the submitted one. A shared secret is returned only if both ciphertexts match, thereby preventing CCAs. However, if a side channel allows observing the re-encryption or the comparison operation, the scheme becomes vulnerable to CCAs again.

Figure 2 shows a simplified version of ML-KEM that relies on the internally defined PKE shown in Figure 1. In Figure 2, re-encryption and comparison operations are highlighted. In Figure 1, we highlight the generation of $\mathbf{r}$; this value is transformed using an NTT – an operation that will become relevant

in Section 4.2; for simplicity, we do not include the NTT or the Inverse Number Theoretic Transform (INTT).

The PKE. The key generation samples a public matrix $\mathbf{A}$ uniformly random but deterministically from a seed ρ ; secrets $\mathbf{e}, \mathbf{s}$ are sampled from a binomial distribution with parameter $\eta = \eta_1$. The public key, given by $\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e}$ and the seed ρ , and the secret key, given by $\mathbf{s}$, are returned as a key pair. The encryption retrieves the public key from ρ , deterministically samples vectors $\mathbf{r}_1, \mathbf{r}, \mathbf{e}_1 \in R_q^k$, and a value $e_2 \in R_q$ from a binomial distribution with parameter η_1 (for $\mathbf{r}_1$) and η_2 (for the remaining vectors and e_2). The bit message m is encoded to a polynomial m_{pol} using the decompression function $\text{Decompress}(x, d) = \lceil xq/2^d \rceil$ with $d = 1$. This means that 0 bits are mapped to $0 \in \mathbb{F}_q$ and 1 bits are mapped to $\lceil q/2 \rceil$. To encrypt the message, $\mathbf{u} = A^\top \mathbf{r} + \mathbf{e}_1 \in R_q^k$ and $v = \mathbf{t}^\top \mathbf{r} + e_2 + m_{\text{pol}} \in R$ are computed and compressed to c_1, c_2 , respectively, giving the ciphertext.

To decrypt a ciphertext back to a message m' , $\mathbf{u}$ and v are recovered from the compressed version given by the ciphertext. Then, m'_{pol} is computed as $\text{Compress}(v - \mathbf{s}^\top \mathbf{u}, 1)$. The compression function is defined as

$$\text{Compress}(x) = \left\lceil x \frac{2^d}{q} \right\rceil \bmod 2^d, \quad (1)$$

i.e., it maps coefficients in $\{0, \dots, q-1\}$ to 0 if closer to 0 than to $q/2$. With high probability, this results in $m = m'$ as

$$v - \mathbf{s}^\top \mathbf{u} = m_{\text{pol}} + \mathbf{e}^\top \mathbf{r} - \mathbf{s}^\top \mathbf{e}_1 + e_2, \quad (2)$$

and the noise term polynomial

$$\mathbf{e}^\top \mathbf{r} - \mathbf{s}^\top \mathbf{e}_1 + e_2 \quad (3)$$

is small.

The KEM. The KEM relies on the PKE: the secret key is amended by the public key, its hash, and a rejection value z . The encapsulation randomly samples a message encrypting it with coins derived from the message and the public key's hash. The future shared secret is derived from the ciphertext, the message's hash and the public key. During decapsulation, the ciphertext is decrypted to recover the message m , allowing the other party to derive the same randomness from m and use the public key to re-encrypt the message into an honestly generated ciphertext $\mathbf{ct}'$. Only if $\mathbf{ct}$ equals $\mathbf{ct}'$ is the shared secret returned.

In the following, we explain and evaluate our and previous attacks on ML-KEM768, i.e., $k = 3$, ignoring the algebraic structure, we see polynomials as vectors of dimension 768 over $\mathbb{F}_q$, and the secret sampled from $\mathbf{B}_\eta$, $\eta = 2$ with the support $\{-2, -1, \dots, 2\}$.

Algorithm 1 PKE.KeyGen**Input:** Seeds ρ, σ **Output:** Secret Key $\mathbf{sk}$, Public Key $\mathbf{pk}$

- 1: $\mathbf{A} \in R_q^{k \times k} \xleftarrow{\$} \mathcal{U}(\rho)$
- 2: $\mathbf{e}, \mathbf{s} \in R^k \xleftarrow{\$} \mathcal{B}_{\eta_1}(\sigma)$
- 3: $\mathbf{t} \leftarrow \mathbf{A}\mathbf{s} + \mathbf{e}$
- 4: **return** $\mathbf{pk} = (\mathbf{t}, \rho), \mathbf{sk} = \mathbf{s}$

Algorithm 2 PKE.Dec**Input:** $\mathbf{sk} = \mathbf{s}, \mathbf{ct} = (c_1, c_2)$ **Output:** Decrypted message m'

- 1: $\mathbf{u}', v' \leftarrow \text{Decompress}(\mathbf{ct})$
- 2: $m_{\text{pol}} \leftarrow v' - \langle \mathbf{s}, \mathbf{u}' \rangle$
- 3: $m' \leftarrow \text{Compress}(m_{\text{pol}}, 1)$
- 4: **return** m'

Algorithm 3 PKE.Enc**Input:** $\mathbf{pk} = (\mathbf{t}, \rho), m, \text{coins coins}$ **Output:** Ciphertext $\mathbf{ct} = (c_1, c_2)$

- 1: $\mathbf{A} \in R^{k \times k} \xleftarrow{\$} \mathcal{U}(\rho)$
- 2: $\mathbf{r}_1 \in R^k \xleftarrow{\$} \mathcal{B}_{\eta_1}(\text{coins})$
- 3: $\mathbf{r} \in R^k \xleftarrow{\$} \mathcal{B}_{\eta_2}(\text{coins})$
- 4: $\mathbf{e}_1 \in R^k \xleftarrow{\$} \mathcal{B}_{\eta_2}(\text{coins})$
- 5: $e_2 \in R \xleftarrow{\$} \mathcal{B}_{\eta_2}(\text{coins})$
- 6: $m_{\text{poly}} \leftarrow \text{Decompress}(m, 1)$
- 7: $\mathbf{u} \leftarrow \mathbf{A}^\top \mathbf{r} + \mathbf{e}_1$
- 8: $v \leftarrow \langle \mathbf{t}, \mathbf{r} \rangle + e_2 + m_{\text{pol}}$
- 9: $c_1 \leftarrow \text{Compress}(\mathbf{u}, d_u)$
- 10: $c_2 \leftarrow \text{Compress}(v, d_v)$
- 11: **return** $\mathbf{ct} = (c_1, c_2)$

Fig. 1: Simplified PKE defined by ML-KEM. The recovery of m' and the sampling of $\mathbf{r}$ are highlighted.

2.3 PC-Oracle Attacks

The terms key mismatch attacks and PC-oracle attacks are not always used consistently. In this work, we call an attack that exploits a side channel to carry out a key mismatch attack a PC-oracle attack. Key mismatch attacks were first introduced by Fluhrer [12] in 2016 and later studied in the context of ML-KEM in [46] and [47], and more recently in [18] and [19]. In [50], Ravi et al. introduced PC-oracle attacks against several lattice-based KEMs. These attacks have since seen many improvements, e.g., in [53, 48, 52]. The authors of [56] showed that even masked implementations may be highly vulnerable. They rely on the observation that one can select a subkey and choose ciphertexts that decrypt to messages depending solely on the subkey. Concretely, if we set $\mathbf{u} = (208x^0, 0, 0) \in R_q^3$ and $v = t \cdot 208 = t \cdot 208x^0 + 0x^1 + \dots + 0x^{n-1}$ for some t , then

$$v - \mathbf{u}^\top \mathbf{s} = (t \cdot 208 - 208s_0, -208s_1, \dots, -208s_{n-1}) \in R_q \quad (4)$$

is compressed, i.e., “recovered” in line 3 of Algorithm 2 (see highlight), to

$$m' = \left(\left\lceil \frac{2}{q} (t \cdot 208 - 208s_0) \right\rceil, 0, \dots, 0 \right) = \left(\left\lceil \frac{416}{q} (t - s_0) \right\rceil, 0, \dots, 0 \right). \quad (5)$$

Let us fix $t = 3$. Then $m'_0 = 0$ implies that $s_0 > -2$, while $m'_0 = 1$ implies that $s_0 \leq -2$. If we let $t = 4$, then $m'_0 = 0$ implies that $s_0 > -1$, while $m'_0 = 1$ implies that $s_0 \leq -1$; and so on. If a side channel allows us to observe the message, then we may learn the value of s_0 and then vary the position of the value in v , and then the positions of the 1 polynomial in $\mathbf{u}$, to recover more coefficients of the components of $\mathbf{s}$. This is called a *binary PC-oracle attack*.

Algorithm 4 KEM.KeyGen**Input:** Randomness seeds ρ, σ **Output:** Public key pk , secret key sk

```

1:  $z \xleftarrow{\$} \mathcal{U}()$ 
2:  $\text{pk}, \text{sk}_{\text{pke}} \leftarrow \text{PKE.KeyGen}()$ 
3:  $h = H(\text{pk})$ 
4:  $\text{sk} \leftarrow (\text{sk}_{\text{pke}}, \text{pk}, h, z)$ 
5: return  $\text{pk}, \text{sk}$ 

```

Algorithm 5 KEM.Encaps**Input:** pk **Output:** Ciphertext ct , shared secret K

```

1:  $m \xleftarrow{\$} \mathcal{U}()$ 
2:  $\bar{K}, \mathbf{r} \leftarrow G((m, H(\text{pk})))$ 
3:  $\text{ct} \leftarrow \text{PKE.Enc}(\text{pk}, m, \mathbf{r})$ 
4:  $K \leftarrow \text{KDF}((\bar{K}, H(\text{ct})))$ 
5: return  $\text{ct}, K$ 

```

Algorithm 6 KEM.Decaps**Input:** $\text{sk} = (\text{sk}_{\text{pke}}, \text{pk}, h, z), \text{ct}$ **Output:** Shared secret K

```

1:  $m' \leftarrow \text{PKE.Dec}(\text{sk}_{\text{pke}}, \text{ct}_{\text{pke}})$ 
2:  $\bar{K}', \mathbf{r}' \leftarrow G((m', h))$ 
3:  $\text{ct}' \leftarrow \text{PKE.Enc}(\text{pk}, m', \mathbf{r}')$ 
4:  $b \leftarrow \text{Compare}(\text{ct}, \text{ct}')$ 
5: if  $b$  then
6:   return  $K = \text{KDF}(\bar{K}', H(\text{ct}))$ 
7: else
8:   return  $K = \text{KDF}(z, H(\text{ct}))$ 
9: end if

```

Fig. 2: Simplified version of ML-KEM. G and H are hash functions. The re-encryption and comparison operations are highlighted.

Parallel PC-Oracles and Tournament oracles. In a *parallel PC-oracle attack*, proposed against ML-KEM in [55], several coefficients of v are set to $t_i \cdot \lceil q/16 \rceil$. The adversary then learns information about all these secret key coefficients in parallel. However, setting N values results in 2^N messages that need to be distinguished using the side channel. In PC-oracle attacks, the side channel is usually abstracted as an oracle that returns N positions of the message, i.e., N inequalities, given a query represented by a vector $\mathbf{a}$ representing the right side of the inequalities.

The work of [47] proposes using a binary decision tree and the authors of [55] and [48] both propose using binary decision trees for parallel PC-oracle attacks, where [48] proposes an optimized tree. In [48], the authors call this a tournament approach, as the side-channel information for messages is compared pairwise, always keeping the current best-matching candidate – essentially a 2^N participant tournament leading to a winning message. This leads to the abstraction used, e.g., in [48]: the oracle returns the correct message with probability α and a random different message with probability $1 - \alpha$.

Query performance and noise resistance. In [18], 1588 queries are required to fully recover the secret key with a binary oracle. Their result is close to the Shannon entropy bound of 1560 queries. However, assuming a key mismatch attack, they do not consider an oracle that can give incorrect answers.

The PC-oracle attack of [48] achieves an accuracy of 1 when recovering the message from side-channel information in a tournament-based setting using traces from a physical attack. In case of higher noise, they refer to the methods of [41,53].

These works propose recovering a partially correct key that is then checked and corrected using additional oracle queries. In [53], recovering an ML-KEM512 key with a few errors using a binary oracle (i.e., $N = 1$) requires over 3000 queries at $\alpha = 0.96$, which is considered low accuracy. For $\alpha = 0.90$, more than 7700 queries are needed with an average of 0.25 incorrect coefficients per key.

The work of [33] handles noisier oracles while requiring fewer queries. They propose using so-called “grafted trees” – a multiphase approach improving upon and sharing similarities with majority voting. Their proposal first attempts to recover a subkey from an imperfect oracle. Then, they construct an improved tree based on the information in the first round. Finally, the subkey is recovered if the results from the first two rounds match; otherwise, they perform an additional round and use majority voting. Targeting $N = 8$ coefficients of ML-KEM1024, they achieve an error rate of 0.351 coefficients per recovered key with 1332 queries at $\alpha = 0.95$; and an error rate of 0.646 coefficients with 2021 queries at $\alpha = 0.90$.

In this work, we show that our soft-analytic approach drastically reduces error rates while dealing with much higher noise levels. For example, for $N = 1$, we can handle $\alpha = 0.51$ with an error rate of 0; for $\alpha = 0.9$, we require merely 10368.0 queries for ML-KEM768 with 0 incorrect coefficients. For $N = 8$ setting and $\alpha = 0.5$, an average of 1113.6 queries at an error rate of 0; it is also 0 for 10000 experiments at $\alpha = 0.1$. Moreover, the number of queries can be further reduced by relaxing the abort criteria and allowing a few errors.

2.4 Soft-Analytic Side-Channel Attacks

Side-channel attacks often recover noisy information on several intermediates. The seminal work of [57] proposes using “soft-analytic” methods to recover the secret key from such information – in contrast to “hard” algebraic methods. Recovery from noisy intermediates under algorithmic knowledge can be viewed as a decoding problem, leading the authors to propose techniques such as BP. These methods can also be combined with algebraic techniques as shown, e.g., in [22] for targeting ML-KEM.

BP was first described in the context of Low-Density Parity Check (LDPC) codes by Gallager [15] and in a more generic way in [42]. For a thorough introduction, we refer to [34]. Given a factorization of the joint density function as a *factor graph*, BP computes or approximates the marginals. A factor graph consists of two types of nodes: *variable nodes* and *factor nodes*. The former represent unknown variables, the latter represent the relations between variable nodes.

For a vector of Random Variable (RV)s, $\mathbf{X} = (X_0, \dots, X_{n-1})$, with joint density $p(\mathbf{X})$ and $f_i : I_i \rightarrow [0, 1] \subseteq \mathbb{R}$ such that $p(\mathbf{X}) = \prod_{i=0}^{n'} f_i(I_i)$ for $n' < n$ and $I_i \subseteq \{X_0, \dots, X_{n-1}\}$, BP approximates the marginals for p , i.e., approximates

$$Z_i(x) = \sum_{\mathbf{x}, x_i=x} p(\mathbf{x}) \quad (6)$$

The components of $\mathbf{X}$ are represented by the variable nodes, the functions f_i by factor nodes.

We denote the message sent from variable node i to factor node j by $p_{i,j}$, and the message from factor node j to variable node i by $q_{j,i}$. The variable nodes compute

$$p_{i,j}(x) \leftarrow \prod_{j', j' \neq j} q_{j,i'}(x). \quad (7)$$

The update from the factor nodes is computed as

$$q_{j,i}(x) \leftarrow \sum_{\mathbf{x}, x_i=x} f_j(\mathbf{x}) \prod_{i', i' \neq i} q_{j,i'}(x). \quad (8)$$

In simpler terms: each node computes the update for a node based on the message from all *other* connected nodes and the relation it represents.

If the graph is acyclic, BP is a reordering of the chain rule and thus is exact. For cyclic graphs, BP is called *loopy belief propagation* and offers only an approximation. In SCA, loopy BP has proven to be a valuable tool, often providing good approximations (see, e.g., [37] for an overview).

In this work, we provide three different approaches to handle side-channel information from an oracle: a computationally intensive optimal updating of the joint density, an efficient but rough approximation, and a trade-off between the two, offered by our BP. Further, we propose an extension to [26] that includes a BP instantiation. However, our main algorithm does not rely on BP; it instead tracks the full joint probability distribution. While not using BP, we argue that this should clearly still be seen as a soft-analytic technique.

3 Exploiting Imperfect PC Oracles

In this work, we propose a soft-analytic [57] approach to PC-oracle attacks. Instead of processing hard information and considering that it could be incorrect, we consider the key coefficients to be random variables and keep track of their probability distributions. Thereby, noise-tolerance and information-theoretic considerations are intrinsic to these attacks. In Section 5, we show that even the naïve updating performs better than non-SASCA approaches. We believe that this is clear evidence for the advantage of soft-analytic techniques.

Oracle abstraction. We adopt the ciphertext construction from previous works, as described in Section 2.3, where *querying the oracle for $\mathbf{a}$* means constructing a chosen ciphertext with N positions being non-zero leading to one of 2^N possible messages, depending on $\mathbf{a}$, and submitting it to the device under attack. The *actual message* m_{actual} is the unique value in $[2^N]$ that fulfills

$$s_i \leq a_i \Leftrightarrow m_i = 1 \quad \forall i \in [N], \quad (9)$$

where s_i denotes secret coefficients and m_i are the corresponding message bits. In our case, we now treat an oracle response as a probability distribution on all possible messages $m \in [2^N]$.

Definition 1. We call a random variable $\mathcal{O}(\mathbf{a}) = (p_0, \dots, p_{2^N-1})$, where $\sum_{j=0}^{2^N-1} p_j = 1$, a distribution oracle and associate with it the probability distribution $\{m : p_m \mid m \in [2^N]\}$.

In the following, by oracle, we mean distribution oracle, and we implicitly assume that the Mutual Information (MI) between oracles and the respective secret key is non-zero. Whenever an oracle returns an actual message and not a probability distribution, we interpret the message as the probability distribution with probability one for that message.

For example, the initial query for $N = 3$ would typically be $\mathbf{a} = (0, 0, 0)$, that is, asking for all three coefficients whether they are smaller than or equal to 0. Then, for a subkey with the first coefficient smaller than or equal to 0 and the next two coefficients positive, $m_{\text{actual}} = 4$, which we, by abuse of notation, also denote as $m_{\text{actual}} = (0, 0, 1)$. A response could be the distribution $\mathcal{D}$ given by

$$P(m = m') = \begin{cases} \frac{1}{20} & m' \in \{0, 3, 5, 6, 7\} \\ \frac{1}{4} & m' \in \{1, 2, 4\} \end{cases}. \quad (10)$$

We denote querying and responding by $\mathcal{D} = \mathcal{O}(\mathbf{a})$.

Terminology. By the *joint probability distribution*, we mean the joint probability distribution of all key coefficients given all (previous) queries. By the *marginals* we mean the marginals of the joint probability distribution, i.e., the distribution for individual key coefficients given all previous queries. We often only work with an approximation of the marginals. *Targeted key coefficients*, or sometimes just key coefficients, refer to the coefficients of the subkey currently being targeted – recall that we iteratively recover subkeys of dimension N .

High-level overview. In all our attacks, we target N coefficients of the secret key in parallel. That means that we query the oracle about the current N positions until we recover them and then proceed to the next block of N positions. We will describe how to recover one such block – the remaining positions are recovered the same way.

We keep track of (approximations of) the marginal distributions of the joint probability of the key coefficients conditioned on all previous oracle responses. These distributions serve as a basis for selecting new queries to the oracle. After each update, we check whether the sum of entropies of all margins is smaller than the *entropy bound* h_{abort} ; in that case we set the key coefficients of the currently targeted block to the most likely values according to our distributions.

In the following, we present three types of update and two types of query selection. The methods range from a naïve approximation of the marginals to their exact computation, of which the latter allows for far better query selection.

3.1 Updating Distributions given Oracle Responses

We first describe how we update our approximation of the marginals given a response to a query; later, we will describe how to select the queries. In all of our

methods, we have a current state that is updated with each query. Note that the updating process is entirely independent of how the queries are chosen, while the query selection depends on the available information.

All methods follow the same reasoning:

Lemma 1. *For a subkey $\mathbf{s}$ and queries $(\mathcal{D}_j = \mathcal{O}(\mathbf{a}_j))_{j \in [K]}$ for that subkey, we have*

$$P(s_i = v \mid (\mathcal{D}_j = \mathcal{O}(\mathbf{a}_j))_{j \in [K]}) \propto P(s_i = v) \sum_{(m_j)_{j \in [2^N]^K}} \prod_{j=0}^{K-1} P_{\mathcal{D}_j}(m_j) \delta_{v \leq a_{j,i}}(m_{j,i}), \quad (11)$$

where the bit value $\delta_A(b)$, for a statement A and a bit b , is 1 if and only if A is true and $b = 1$ or A is false and $b = 0$.

Naïve Updating of Marginals. In each query, we get information about each of the N targeted key coefficients. A simple and inexpensive approach is to store N probability distributions – one for each targeted key coefficient – and update them independently. We compute

$$P(s_i = v \mid (\mathcal{D}_j = \mathcal{O}(\mathbf{a}_j))_{j \in [K]}) \approx P(s_i = v) \prod_{j=0}^{K-1} \sum_{m=0}^{2^N-1} \delta_{v \leq a_{j,i}}(m_i) P_{\mathcal{D}_j}(m). \quad (12)$$

Clearly, this disregards the fact that the inequalities given by the oracle response are dependent, and, thus, loses a substantial amount of information. However, this approach can be realized very efficiently: Denoting the current probabilities for coefficient i by $p_i(v)$, we initialize all $p_i(v)$ to the prior, i.e., $\mathcal{B}_\eta(v)$ for ML-KEM. For each query response $\mathcal{D}$, we update $p_i(v)$ by computing

$$p_i(v) \leftarrow p_i(v) \sum_{m=0}^{2^N-1} \delta_{v_i \leq a_i}(m_i) P_{\mathcal{D}}(m). \quad (13)$$

We continue querying the oracle and updating the approximations to the marginals until the sum of their entropies is smaller than h_{abort} .

For each of the $2\eta+1$ possible values that a secret entry can take, updating each of the N positions according to (13) computes a sum with 2^N terms. Assuming that we need K queries to recover the block of N positions, then the total number of terms adds up to $(2\eta+1)KN2^N$. While exponential in N , as N is generally a small value in practice, the attack is cheap.

Using Belief Propagation. BP, see Section 2.4, allows computing approximations to the marginals of a joint probability distribution given a factorization of the joint mass function. In our case, we may see the message of a query response as a variable node, where the actual message is the true value. The query gives N factor nodes that are “incomplete” inequalities as they depend on the input of the

message nodes and are connected to the variable nodes representing the secret key. An example of such a graph is given in Figure 3 for a three-dimensional subkey and two queries; note that we represent inequalities as smaller-equal and message nodes represent a potential multiplication by -1 and adding 1 to the right side, essentially turning them into the respective greater-than inequality.

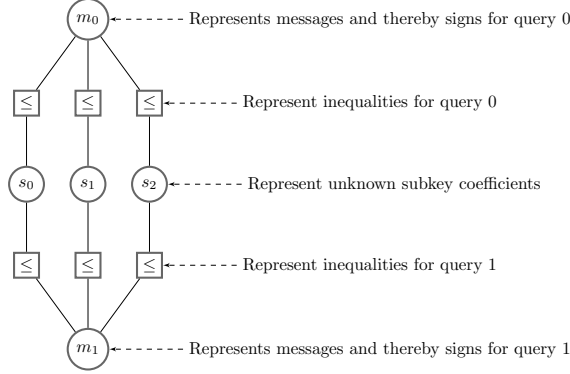


Fig. 3: Graph for three coefficients and two queries: The inequality nodes represent smaller-equal inequalities for some value $a_{j,i}$ for query j and coefficient i on the right side, and the secret key on the left side. The message nodes represent the query response, potentially changing inequalities to greater-than.

Initialization and update computation. Our graph features one type of non-trivial factor nodes: “incomplete” inequalities. These hold the values of the coefficients’ query, a_i , and are connected to the message nodes, from which the inequalities receive a distribution on the messages. The i -th bit represents the probability of the i -th inequality being a smaller-equal inequality. The variable nodes for the secret subkey coefficients are implicitly connected to factor nodes representing the prior⁹. The prior for the message nodes is the query response $\mathcal{D}_j$.

We denote the probabilities coming from an inequality with index i at query j in a step as follows: the probabilities for subkey coefficients are denoted p_i , the probabilities for messages as q_j . The update for the subkey coefficients is computed as

$$p_i(v) \propto q_j(1)\delta_{v \leq a_i}(1) + q_j(0)\delta_{v > a_i}(0) = \begin{cases} q_j(1) & \text{if } v \leq a_i \\ q_j(0) & \text{if } v > a_i \end{cases}. \quad (14)$$

The update for the message nodes, coming from inequality i , is computed as

$$q_j(1) \leftarrow \sum_{v=0}^{a_i} p_i(v) \text{ and then clearly } q_j(0) = \sum_{v=a_i+1}^{\eta} p_i(v). \quad (15)$$

⁹ We choose to make priors implicit to variable nodes in an effort to reduce notations.

Abort criteria and integrating new queries. From the BP, we obtain approximation for marginals after every single step. Again, we abort once the sum of entropies is below h_{abort} . Note that we have to restart the BP with every newly obtained query; this leads to a linear overhead compared to the other two methods that can update their probabilities without having to recompute the complete state. One viable strategy for an attacker would be to obtain as many queries as possible and then run BP once to recover the secret key.

Intuitive description and iterative approach. Without knowing about BP, an intuitive approach could be to just use Bayes' formula on the current state. The state is initialized with the prior – just as in BP. After each query, we compute the probabilities for each given message based on the current state, i.e., the current prior. We may then update the distributions of the key coefficients using Bayes' formula as normalized product of the current prior and the messages conditioned on the key. Then, after the update, a new query is submitted.

The approach is similar to the BP instantiation with a maximum number of iterations of 1 and restarting after every query. Using the BP with a larger number of iterations but without restarting offers an option to reduce the computational cost by a linear factor; however, it may increase the number of required queries.

Complexity. Variable nodes representing secret key coefficients compute K products of K message; however, this can be done in $2K(2\eta + 1)$ multiplications (see, e.g., approaches from [44,25,22]) leading to a cost of $2NK(2\eta + 1)$. The factor nodes have only two inputs and have a total cost of $3NK(2\eta + 1)$ multiplications and additions and $2^N NK(2\eta + 1)$ inequality checks. Finally, the query nodes, representing the messages, compute $K(2N(2\eta + 1) + N)$ multiplications.

Optimal Attacks. Both using only the marginals and using BP clearly lose information. While BP can provide a reasonably good approximation of the marginals, it is not exact as the factor graph is not acyclic. Instead of approximating the marginals, we can compute the joint probability distribution and obtain the exact marginals after every update. Unfortunately, this requires storing a table of size $(2\eta + 1)^N$. Nevertheless, as seen in Section 5, this is a reasonably fast (in terms of runtime) attack that may outperform BP for up to 4-dimensional subkeys¹⁰.

To keep track of the joint probability distribution, we store the probability of every N -tuple of subkey values, i.e., of every possible subkey. These are initialized using the prior, i.e., $\mathcal{B}_\eta$ in ML-KEM, denoted by p_{prior} . Denoting the probability for a subkey $\mathbf{v} = (v_0, \dots, v_{N-1})$ by $p(\mathbf{v})$, given a query response $\mathcal{D}$ for $\mathbf{a}$, we compute the updated probabilities as

$$p(\mathbf{v}) \leftarrow p(\mathbf{v}) \sum_{m=0}^{2^N-1} P_{\mathcal{D}}(m) \prod_{i=0}^{N-1} \delta_{v_i \leq a_i}(m_i). \quad (16)$$

We then get

¹⁰ Note that our implementations are not highly optimized.

Lemma 2. For K , $(\mathcal{D}_j = \mathcal{O}(\mathbf{a}_j))_{j \in [K]}$, the marginals of p , computed according to (16), equal the conditional probabilities $P(s_i = v \mid (\mathcal{D}_j = \mathcal{O}(\mathbf{a}_j))_{j \in [K]})$.

Complexity. Naïvely Updating $p(\mathbf{v})$ according to (16) costs $N2^N$ inequality checks. For a block of N positions, we must perform this update for $(2\eta + 1)^N$ different values of $\mathbf{v}$. Assuming K queries are needed to recover the entire block, the total number of operations to recover N positions is $K(2\eta + 1)^N N2^N = KN(4\eta + 2)^N$. While this scales badly with N , the attack works quickly up to 5 dimensions. Moreover, it allows us to compare the more efficient naïve and BP-based methods with using the query output in an information-theoretically optimal way.

Intuition on the sub-optimality of the marginal approach. While the marginals clearly hold less information than the full joint distribution, we would like to provide some intuition: Figure 4 shows an example where the entropy of the full joint distribution is already greatly reduced, yet the marginals remain unchanged. This is due to the oracle queries sometimes corresponding to logical equivalences between inequalities in different variables, e.g., equalities. These cannot be captured by the marginals.

		s_1				
		-2	-1	0	1	2
s_0	-2	0	0	0	0	0.0625
	-1	0	0	0	0.25	0
	0	0	0	0.375	0	0
	1	0	0.25	0	0	0
	2	0.0625	0	0	0	0

Fig. 4: Example of a joint distribution for two targeted key coefficients in which the marginals have not changed, but the entropy is greatly reduced – $s_0 = s_1$.

Combining with lattice reduction. Both the BP and the optimal approach can be combined with lattice reduction as proposed in [22]. In BP we have only the marginals while in the optimal approach we know the full joint probability distribution, which may enable improved hint integration following [6,7,36].

3.2 Choosing Oracle Queries

Apart from updating (approximations of) probability distributions, the adversary must select queries. A query is a chosen ciphertext, modeled as a vector $\mathbf{a}$

representing $s_i \leq a_i \Leftrightarrow m_i = 1$. These chosen ciphertexts lead to one of 2^N messages $\mathbf{m}$ during decapsulation – depending on inequalities in the secret key. These messages, which the adversary attempts to distinguish using a side channel, determine the randomness used for re-encryption (see Algorithm 6). The oracle response is modeled as a probability distribution over $\mathbf{m}$. All previous queries are again denoted as $(\mathcal{D}_j = \mathcal{O}(\mathbf{a}_j))_{j \in [K]}$.

In general, the goal is to maximize the MI between the oracle given our query and the secret key, both conditioned on all previous queries. For a perfect oracle¹¹, this is exactly the entropy of the oracle query conditioned on previous queries, as

$$I(\mathbf{s}; \mathcal{O}(\mathbf{a}) \mid (\mathcal{D}_j = \mathcal{O}(\mathbf{a}_j))_{j \in [K]}) = I(\mathcal{O}(\mathbf{a}); \mathbf{s} \mid (\mathcal{D}_j = \mathcal{O}(\mathbf{a}_j))_{j \in [K]}) \quad (17)$$

$$= H(\mathcal{O}(\mathbf{a}) \mid (\mathcal{D}_j = \mathcal{O}(\mathbf{a}_j))_{j \in [K]}) - H(\mathcal{O}(\mathbf{a}) \mid (\mathcal{D}_j = \mathcal{O}(\mathbf{a}_j))_{j \in [K]}, \mathbf{s}), \quad (18)$$

which is $H(\mathcal{O}(\mathbf{a}) \mid (\mathcal{D}_j = \mathcal{O}(\mathbf{a}_j))_{j \in [K]})$ as $\mathcal{O}(\mathbf{a})$ is fully known if we know $\mathbf{s}$. For a query $\mathbf{a}$, define $A_m(\mathbf{a})$ to be the region cut out by the inequalities that $\mathbf{a}$ and m represent, i.e.,

$$A_m(\mathbf{a}) = \left\{ \mathbf{v} \in \{-\eta, \dots, \eta\}^N \mid (-1)^{m_i} v_i > (-1)^{m_i} a_i - m_i \ \forall i \in [N] \right\}, \quad (19)$$

where m_i denotes the bits of m . Then the entropy $H(\mathcal{O}(\mathbf{a}) \mid (\mathcal{D}_j = \mathcal{O}(\mathbf{a}_j))_{j \in [K]})$ is given by

$$H(\mathcal{O}(\mathbf{a}) \mid (\mathcal{D}_j = \mathcal{O}(\mathbf{a}_j))_{j \in [K]}) = \sum_{m=0}^{2^N} \sum_{\mathbf{v} \in A_m} -p(\mathbf{v}) \log_2 \left(\sum_{\mathbf{v} \in A_m} p(\mathbf{v}) \right), \quad (20)$$

where $p(\mathbf{v})$ denotes the current joint probability distribution, i.e., the current “state”, $p(\mathbf{v}) = P(\mathbf{s} = \mathbf{v} \mid (\mathcal{D}_j = \mathcal{O}(\mathbf{a}_j))_{j \in [K]})$.

For an imperfect (distribution) oracle, the same reasoning applies: The entropy conditioned on the secret key is for an imperfect oracle – by definition – not zero. However, in our abstraction, we assume no further information on the nature of the imperfectness of the oracle, i.e., on $H(\mathcal{O}(\mathbf{a}) \mid (\mathcal{D}_j = \mathcal{O}(\mathbf{a}_j))_{j \in [K]}, \mathbf{s})$. Thus, by (17)-(18) we have to maximize $H(\mathcal{O}(\mathbf{a}) \mid (\mathcal{D}_j = \mathcal{O}(\mathbf{a}_j))_{j \in [K]})$, which means maximizing (20). In the following, we discuss this for all three attack types. In the naïve and BP-based attacks, only the marginals of the joint distribution are available. In the optimal attack, however, the adversary can choose much better queries. Thus, we get the following statement:

Lemma 3. *Given queries $(\mathcal{D}_j = \mathcal{O}(\mathbf{a}_j))_{j \in [K]}$, the MI between the secret and the oracle is maximized by maximizing (20).*

In the following, we will refer to queries/splits according to Lemma 3 as optimal. By this, we mean optimal for a strategy that

- only targets N positions at a time as opposed to the blocks with the highest entropy,

¹¹ Here we write the perfect oracle as a distribution oracle that assigns m_{actual} the probability 1.

- only considers the maximal entropy reduction for the next query,
- and only queries the oracle for inequalities along one of the secret key axes; see [52] for a parallelized mismatch attack that performs other splits.

Query selection approaches. Depending on the attack, our “state” consists of either an approximation of the marginals or a full joint probability distribution. In the former case, we may rely solely on the marginals while in the latter we can achieve better results.

Using the marginals. For a perfect binary oracle, prior work [47] (as pointed out in [18]) selected each a_i such that $\sum_{v=-\eta}^l p_v - \sum_{v=l+1}^{\eta} p_v$ is minimized. In fact, our reasoning above formalizes and generalizes this way of selecting queries. Given an imperfect oracle, we may use the marginal distributions as an approximation of the joint probability distribution – essentially assuming independence – and apply the heuristic that follows Lemma 3. Thus, we choose the query for each position independently so that the entropy of the approximated split distribution is maximized.

Using the joint probability distribution. For optimal updating, we iterate over all queries $\mathbf{a} \in \{-\eta, \dots, \eta - 1\}^N$ and select the one that maximizes the entropy of the split distributions. The drawback is the increased computational cost: $(2\eta)^N$ possible splits must each iterate over $(2\eta + 1)^N$ values in the joint distribution, resulting in $(2\eta)^N (2\eta + 1)^N = (4\eta^2 + 2\eta)^N$ additions. Each split also requires an entropy computation involving 2^N logarithms. Although asymptotically insignificant, this step can take up a substantial part of the running time for the practical range of N . Finding more efficient methods for optimal splits as well as approximations is left for future work.

Intuition on the sub-optimality of using the marginals. It is not obvious that choosing queries based on the marginals gives suboptimal results compared to the method described in this section. Consider the example given in Figure 5: the left side shows an optimal split, while the right shows the marginal-based split.

With marginal splits, two messages provide no information, whereas in the optimal case, all but one message holds information. Marginal splits reveal at most 1 bit of information: a perfect oracle gives 1 bit of information – two messages cannot occur and the other two exclude half the possible values. While positive probabilities may be assigned to the impossible messages, as they are known to be zero, they have no effect in the updating. For an optimal query, the split distribution has the probabilities 0.5, 0.25 and 0.25. Thus, for this query, an perfect oracle reveals 1.5 bits of information.

Choosing targeted positions adaptively. Our methods target a block of N positions until all corresponding secret values are determined with sufficient certainty. Alternatively, the adversary could adaptively choose which positions to target, with – again – MI as the rationale behind: the less information they have about a position, the more they may gain from querying the oracle about it.

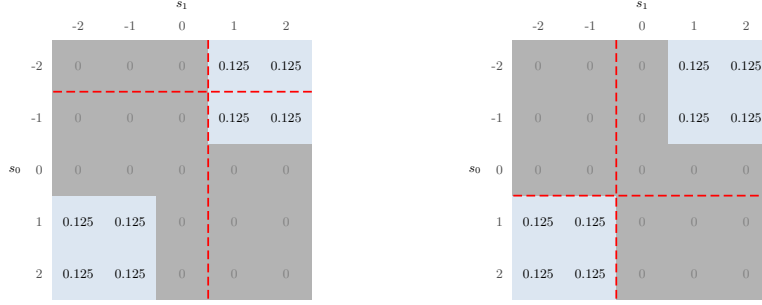


Fig. 5: Joint distribution given previous queries, with an optimal split according to the joint distribution (left) and a split obtained from the marginals (right).

3.3 Joint Distribution Hints

Distribution hints [26] offer a rather generic way to exploit side-channel information linear in the secret key. In our setting, however, they cannot be used, since both distribution hints and previous hint definitions (e.g., [6]) assume *independent* linear information, whereas we obtain *dependent* linear hints in the secret key. To overcome this limitation, we propose *joint distribution hints*:

Definition 2. Let $(\mathbf{w}_j)_{j \in [L]} \in (\mathbb{R}^N)^L$, where N is the dimension of the secret key $\mathbf{x} = (\mathbf{e}, \mathbf{s})$, $\mathcal{D}$ be a probability distribution on $\mathbb{R}^L$, such that

$$(\mathbf{w}_j^\top \mathbf{x})_{j \in [L]} \sim \mathcal{D}. \quad (21)$$

We then call the tuple $((\mathbf{w}_j)_{j \in [L]}, \mathcal{D})$ a joint distribution hint.

This definition is a strict generalization of distribution hints: distribution hints correspond to $L = 1$, while $L \geq 2$ with a distribution $\mathcal{D}$ that does not factor into independent distributions on each coefficient captures information that distribution hints do not cover. Note that when obtaining multiple joint distribution hints, L need not be the same for each hint.

BP-based solver. Joint distribution hints may be solved using BP on a similar factor graph as in [26]: Each secret key coefficient is represented by a variable node. A single joint distribution hint is represented by a factor node that is connected to all variable nodes for which any $w_{i,j}$ is non-zero. The factor nodes compute the distributions of $\sum_{i, i \neq k}^{N-1} w_{j,i} s_i$ for all $k \in [L]$ and $j \in [L]$. Denote the probabilities of these distributions by $q_{j,k}(v)$. To update the belief for key coefficient i , denoted by $p_i(v)$ for $s_i = v$, we compute

$$p_i(v) \leftarrow \sum_{\mathbf{a} \in \text{supp } \mathcal{D}} P_{\mathcal{D}}(\mathbf{a}) \prod_{j=0}^{N-1} q_{j,i}(a_j - v), \quad (22)$$

thereby updating the probability for $s_k = v$ using

$$\left(\sum_{i=0}^N w_{j,i} s_i \right)_j = \left((w_{j,k} s_k) + \left(\sum_{i=0, i \neq k}^N w_{j,i} s_i \right) \right)_j \sim \mathcal{D} \quad (23)$$

by computing the distribution of $(\sum_{i=0, i \neq k}^N w_{j,i} s_i)_j$ from the incoming messages – the current belief in the (other) variables. The latter computation can be implemented in $O(N \log(N))$ using a Fast Fourier Transform (FFT) and a forward-backward iteration as described in [26].

For joint distribution hints, the solver described here gives a BP instantiation equivalent to, though less straightforward than, the one in Section 3.1. For evaluations, we use an easily accessible Python implementation, and additionally provide a highly optimized, generic Rust version based on the open-source implementation of [26]. For hints with low information content, a filtering technique as in [23] may be necessary. Further, the BP, as well as the greedy solver below, may be improved using subsequent lattice reduction as described in [22,26].

Greedy solver. Derived from the BP instantiation similarly to [26], a greedy solver stores a guess, scores the potential changes of the guess, then performs the highest-scored changes. For joint distribution hints, the score for changing coefficient i by $+c$ can be computed as $s_i(c) = \sum_{a \in \text{supp } \mathcal{D}} P_{\mathcal{D}}(a) \prod_{j=0}^N |\mathbf{v}_{j,i}^\top \mathbf{x}' + v_{j,i}c - a|$. Thus, the computationally intensive calculation of N partial sums of $(N-1)$ random variables per hint is avoided and replaced by computing a distance. The implementation and evaluation of a greedy solver is left for future work.

4 Oracle Instantiations

Many different oracle instantiations can be derived from concrete side-channels. Prior work focused mostly on perfect or tournament-based oracles – the latter abstracting real-world side-channels well but often losing information when instantiating. We introduce two oracles – based on the noisy HW model – that are closer to real attacks, providing less abstraction but exploiting more information and better suited for high-noise scenarios. We also discuss the information-theoretic aspects of tournament-based oracles, deriving a lower bound on the required number of queries. In Section 5, we compare our results against previous attacks and this lower bound.

4.1 Tournament-Based Oracle

Recall from Section 3 that a tournament-based oracle assumes that the adversary assigns scores to messages – e.g., via a template attack – and compares them pairwise, discarding lower-scored ones, i.e., not all messages are being compared. Therefore, a message is selected as the assumed actual message after $N-1$

comparisons, where N is the number of targeted coefficients, resulting in efficiency even for many positions. This approach has been used for measurements of a physical device in [48]. A tournament-based oracle, $\mathcal{O}_{\text{tou},N,\alpha}$, is instantiated by its correctness probability α , the probability that the returned message is correct. In simulations, the adversary receives the correct message, m_{actual} , with probability α , and a random incorrect message from $[2^N]$ with probability $1 - \alpha$.

Information-theoretic lower limit. Lemma 3 shows how to optimally choose queries. For a tournament-based oracle, the mutual information $I(\mathcal{O}_{\text{tou},N,\alpha}; \mathbf{s})$ can be computed in closed form. We first derive an upper limit on how many bits of information an oracle can maximally output, which in turn gives a lower limit on the number of queries. Targeting N coefficients, we obtain an inequality per coefficient, and at most N bits. The entropy of the oracle response is

$$H(\mathcal{O}_{\text{tou},N,\alpha}) = -\alpha \cdot \log_2(\alpha) - (2^N - 1) \frac{1 - \alpha}{2^N - 1} \log_2 \left(\frac{1 - \alpha}{2^N - 1} \right) \quad (24)$$

$$= h(\alpha) + (1 - \alpha) \cdot \log_2(2^N - 1) \approx h(\alpha) + (1 - \alpha) \cdot N, \quad (25)$$

where the approximation holds well for large N . For the case $N = 1$, the oracle is simply a binary symmetric channel. The minimum number of queries to recover the whole secret then becomes

$$\left\lceil \frac{H(\mathbf{s})}{N - (h(\alpha) + (1 - \alpha) \cdot \log_2(2^N - 1))} \right\rceil \approx \left\lceil \frac{H(\mathbf{s})}{\alpha \cdot N - h(\alpha)} \right\rceil, \quad (26)$$

where $H(\mathbf{s})$ is the entropy of the whole secret key.

Table 1: Minimum number of queries per targeted positions N and α .

N/α	0.9	0.8	0.7	0.6	0.5	0.4	0.3	0.2	0.1
1	2936.95	5608.37	13137.42	53685.46	∞	53685.46	13137.42	5608.37	2936.95
2	1136.26	1622.69	2424.57	3947.54	7515.13	19975.57	168865.37	154379.88	14918.29
4	496.62	624.64	801.14	1063.59	1490.16	2276.97	4062.49	10222.45	105353.27
8	231.67	274.6	330.38	407.05	519.35	698.58	1024.21	1767.0	4640.27
10	182.8	214.27	254.86	310.07	389.81	514.71	735.73	1219.15	2929.95

Information loss. The tournament-based oracle ignores that a side channel usually results in a probability distribution for the targeted values, which in turn gives a probability distribution for the messages. Returning only the most likely value and discarding the rest of the distribution is clearly suboptimal. For instance, measurements that show a uniform distribution with a tiny bias towards one value are treated similarly as measurements that single out one message with probability one. With our definition of distribution oracles, this loss of information is avoided.

4.2 Noisy Hamming Weight Oracle

PC-oracle attacks allow for on-device profiling and template attacks [5] without requiring a clone device. In these attacks, the adversary learns a probability distribution from each measurement; this type of soft information is the motivation behind distribution oracles and our oracle instantiation allows for more efficient attacks.

Online template attacks. In (online) template attacks, the adversary first chooses a leakage model, usually including Gaussian noise. This means that, for some α , σ , and $\mathcal{L}$, a processed intermediate value u leaks

$$\alpha\mathcal{L}(u) + \mathcal{N}(0, \sigma). \quad (27)$$

The leakage function $\mathcal{L}$ is often assumed to be the HW – the model is referred to as the noisy HW model. In the following, we assume $\alpha = 1$ for simplicity. The adversary then sends in chosen ciphertexts for which they know the resulting message and records the distribution at a Point of Interest (PoI) for intermediate values u , learning the expected value and standard deviation σ for given intermediates. Given a measurement x at the PoI, they may now match against the templates and recover probabilities for each intermediate as

$$P(u = u' \mid x) = \frac{p(x \mid u = u')P(u = u')}{\sum_{u''} P(x \mid u = u'')} = \frac{p_{\mathcal{N}(\alpha\mathcal{L}(u), \sigma)}(m)P(u = u')}{\sum_{u''} P(x \mid u = u'')}, \quad (28)$$

by Bayes' rule. In turn, as the relation between messages and intermediates is public, the probabilities on intermediate values occurring during the re-encryption map to probability distributions on messages.

Noise HW oracle instantiation. To instantiate our model of an oracle, $\mathcal{O}_{\text{nhw}, N, r, \sigma}$, we assume a template attack on the last layer of the NTT in Algorithm 3 (see highlighted line). These $k \cdot n$ values, i.e., 512, 768, or 1024 are distributed uniformly random over $\mathbb{F}_q$. Concretely, the adversary is assumed to carry out the following attacks:

1. Choose r values to be targeted.
2. Submit honestly-generated ciphertexts to record a template for each of the r values.
3. For a query $\mathbf{a}$, submit the corresponding ciphertext and perform measurements of all r values.
4. Compute a probability distribution for each of the values of r based on the templates and measurements.
5. Assign each possible message the normalized product of the probability of the intermediates corresponding to the message.

Computing the leaked information involves Gaussian mixture models over HW distributions, which have no closed form expression [28]. Our implementation offers a routine to evaluate the leaked information per oracle call on a fresh secret key or during complete attacks. However, for the optimal attack, this information is entirely contained in the number of oracle calls.

Masked Noisy HW oracle instantiation. Masking [4] is one of the most notable countermeasures against side-channel analysis, and proposals for masked ML-KEM have been published [3,21]. Arithmetic masking shares a value $x \in \mathbb{F}_q$ as $x = \sum_{i=0}^t x_i$, where x_i are uniformly random over $\mathbb{F}_q$ and t is the masking order. The NTT is linear and can thus be computed on each share individually. Consequently, we assume that the adversary targets the $t + 1$ shares of each of the targeted intermediate values. They then retrieve the distribution of the intermediate as the convolution (modulo q) of the distributions of the shares. From the distribution on the intermediates, they then derive the distribution on the messages as before, giving an oracle $\mathcal{O}_{\text{shw},N,r,\sigma}$.

Relevance of exploiting all information. In unmasked implementations or under insufficient noise, abstracting the noisy HW model as a tournament-based oracle may often yield high accuracies. With masking, the information given on the secret intermediate value by measuring the shares decreases exponentially in the masking order (see, e.g., [10] for a thorough analysis with practical results). This may be less noticeable for low-noise few-shares settings, e.g., on evaluation devices. However, in real-world settings under sufficient noise, attacks are unlikely to succeed if much of the given information is ignored – even for distinguishing attacks, which are particularly hard to defend against. This is especially true when targeting arithmetic masking, which often behaves more favorably [35,23].

We may instantiate a tournament oracle based on the noisy HW model. This allows us to assess the loss in information caused by fitting leakage into an abstract tournament-based oracle. To evaluate the cost of abstraction, we 1) sample a fresh key, 2) issue the initial query to the oracle, 3) perform a tournament for the queries, and 4) record a success if it matches m_{actual} , a failure otherwise. We set α to the accuracy over 100 samples, as shown in Figure 6, yielding an optimal instantiation of a tournament-based oracle from this type of leakage. For low noise or without countermeasures, α can be very high, close to or even equal to 1.0. However, with masking and moderate noise, or when only few positions can be targeted (e.g., due to hiding countermeasures), α quickly drops. Together, Figures 6 to 8 show that instantiating a tournament-based oracle from noisy HW leakage requires far more traces.

5 Evaluation

We evaluate the proposed attacks against the three proposed oracle instantiations. Our attacks are denoted “optimal” for the full joint distribution, “bp” for the BP approximation, and “naïve” for the naïve approximation. Under the assumption of Lemma 3, the optimal attack exploits the given information optimally. Thus, we do not provide a separate information-theoretic analysis – this is implicitly given by the optimal attack. Our evaluations show the number of traces for recovering the full key in ML-KEM768 (1024 in Table 2), i.e., $T \cdot \frac{768}{N}$ where T is the number of traces to recover an N -dimensional subkey.

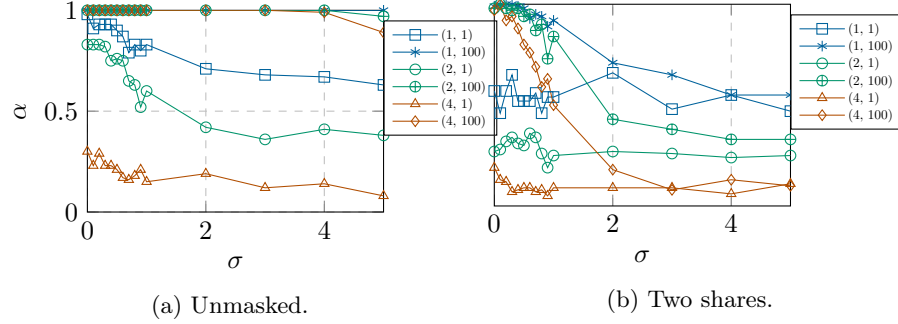


Fig. 6: Comparing tournament and noisy HW-based models: for a given σ , we achieve α if we instantiate a tournament-based oracle using a noisy HW oracle for an unmasked setting and two shares; graphs for $(N, \text{targeted NTT values})$.

Methodology and implementation. Our attacks are implemented in Python and rely on PQClean [30] for an ML-KEM implementation. We always first sample a random secret key using the de-randomized key generation routine with a random seed. Each oracle call first obtains the actual message m_{actual} for the submitted chosen-ciphertext. Then, in the tournament-based oracle, we obtain randomness in $[0, 1]$ and return m_{actual} if it is smaller than α ; otherwise, we rejection-sample a different uniform message. The returned message is assigned probability α , all other messages are assigned probability $\frac{1-\alpha}{2^N-1}$. The noisy HW oracles sample leakage on the targeted intermediate NTT value(s) or their shares, perform a template attack, then return the resulting distribution.

All attacks can be combined with any oracle instantiation and the optimal attack also allows for different types of split selection. However, we only evaluate the optimal, w.r.t. the assumptions in Section 3.2, split selection. Despite not optimizing our attacks, they can still be run on a laptop for a reasonable number of targeted positions N .

For our evaluations, we target the first N coefficients 10 times per setting. Exceptions are the failure evaluations, Table 2, and Figure 6 from the previous section, for which we used 10000, 100, and 100 samples per setting, respectively.

Abort criteria and failure rate. We abort both the BP and the optimal attacks if the marginals have entropy below 0.001, or after a maximum of 10000 oracle calls for all settings except the binary tournament-based oracle at $\alpha = 0.51$, where we allow up to 100000 calls. This results in very low failure rates, typically zero; for instance, the BP had one to four failures in the tournament-based setting for $\alpha \in \{0.9, 0.95\}$, $N \in \{4, 8\}$ across 10000 samples, while the optimal attack had no failures for $\alpha \in \{0.9, 0.95\}$, $N = 4$. Failing examples are excluded from the computation of the average number of oracle calls, as an adversary would likely detect when an excessive number of calls yield little information while we continue iterating up to 10000 calls.

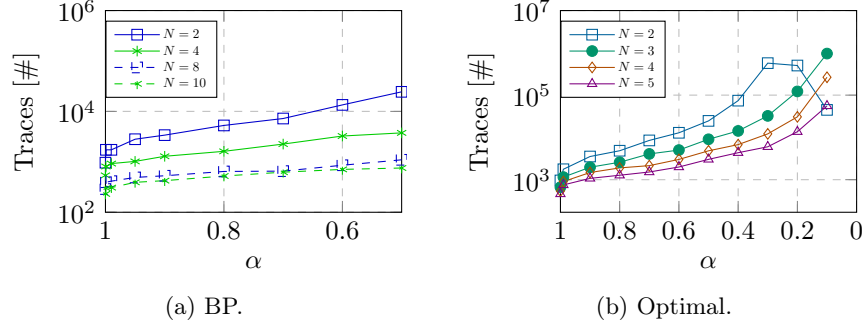


Fig. 7: Results for BP and the optimal attack targeting a tournament-based oracle in terms of number of calls per oracle accuracy α . Note that the least information occurs at $\alpha = \frac{1}{2^N}$, which explains the plateau for $N = 2$.

Table 2: Comparison against [33] for $N = 8$ targeting ML-KEM1024.

α	0.95	0.9	0.8	0.7	0.6	0.5
[33]	1336.8	2041.6	4625.8	7295.8	n.a.	n.a.
This work	623.36	706.56	817.92	944.64	1166.08	1529.6

The same entropy bound gives different guarantees depending on the attack. Therefore, in some settings, the BP yields very comparable results. However, it should be noted that the BP failure rate, although low, is still higher than that of the optimal attack. The optimal attack can be improved in terms of the number of oracle calls by allowing a higher error rate, i.e., aborting earlier.

Tournament-based oracle. We evaluate the tournament-based oracle for $\alpha \in \{0.1, \dots, 0.99\}$ using both BP and the optimal attack, with results shown in Figure 7. The BP becomes excessively slow when many queries are needed, so we evaluate it only for settings with fewer queries and leave further optimization to future work. For a binary oracle and the optimal attack, a mean of 19319142.912 queries is required for a success rate of 1 at $\alpha = 0.51$ (10,000 samples), while at $\alpha = 0.55$, 722380.8 queries are needed (10 samples).

The most recent proposal [33] targets imperfect PC-oracles with α as low as 0.7 for N up to 8, outperforming previous works. Thus, we compare our attacks to theirs, focusing on the BP due to the high computational cost of the optimal attack for $N = 8$. Table 2 provides results for $N = 8$ for different values of α , showing that our method requires fewer traces while achieving zero error.

Noisy HW Oracle. The noisy HW oracle depends on the dimension N of the targeted subkey, the number of targeted NTT values, and σ . The masked noisy HW oracle also depends on the number of shares $t + 1$. In the unmasked setting, Figure 8a, we evaluate the optimal attack for σ up to 10 and $N \in \{1, \dots, 4\}$;

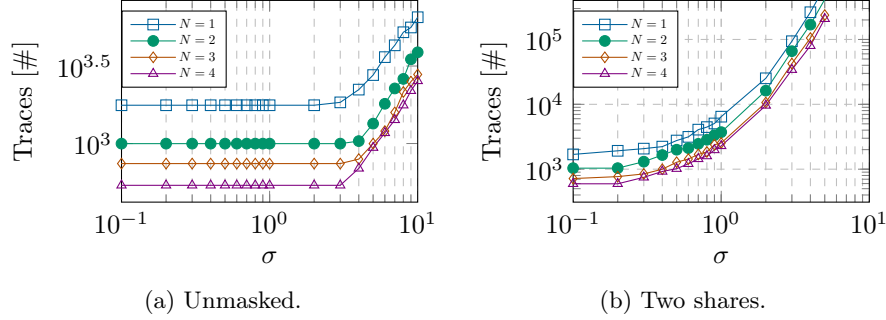


Fig. 8: Results in terms of oracle queries per noise level for an unmasked noisy HW oracle and a first-order masked noisy HW oracle targeting 100 NTT values.

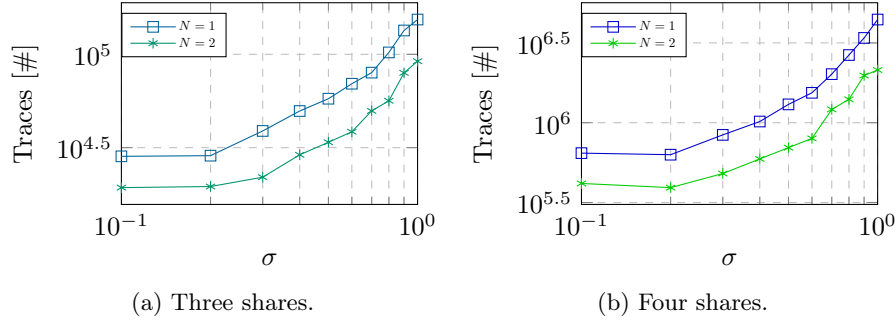


Fig. 9: Results in terms of oracle queries per noise level for second and third-order masked noisy HW oracle targeting 100 NTT values.

for the BP and the naïve attack, we additionally evaluate for N up to 10 (see Figure 10). For the masked setting (see Figures 8b and 9), we observed a single case in third-order masking with $\sigma = 1$ where the subkey was not recovered after 10000 oracle calls using an entropy bound of 0.01.

Targeting more than 5 positions quickly becomes infeasible with the full updating. However, both the BP and the naïve attack can easily handle this as shown in Figure 10 for $\sigma \in \{0.1, 0.2, \dots, 1, 2, \dots, 5\}$ and $N \in \{1, \dots, 10\}$ for 100 targeted values. The success rate for the BP approach is always 1. For the naïve approach, it is 0.8 at $N = 10$ and $\sigma \geq 4$, and 1 otherwise. The mean computation time for different settings lies between 22 and 1492 seconds.

6 Conclusion

In this work, we present several PC-oracle attacks based on soft-analytic techniques, approximating or computing the marginals of the joint distribution of the targeted subkey. The attacks exhibit significantly higher noise tolerance. Furthermore, we introduce a new oracle abstraction that allows for further exploiting

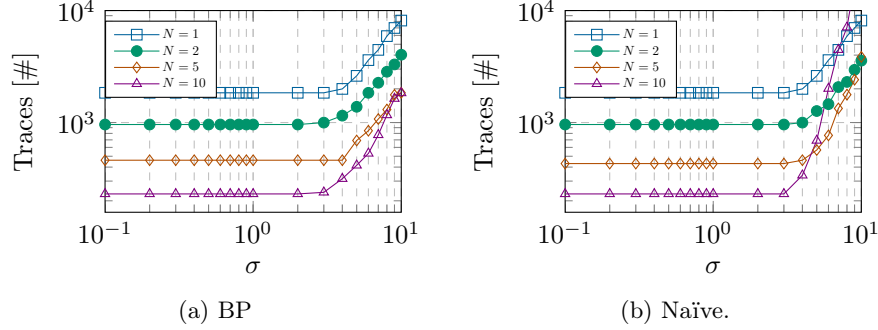


Fig. 10: Targeting up to 10 positions and 100 NTT values.

available information. Using our oracles, we discuss instantiations that more closely reflect protected real-world implementations.

The results allow us to draw several conclusions. First, PC-oracle attacks can be highly noise tolerant. In combination with the FO transform offering a large attack surface, this makes them even more threatening. Secondly, even rather naïve soft-analytic techniques may outperform different, more sophisticated approaches. This observation is in-line with previous works [16] despite limitations [54]. Our work further motivates exploring SASCAs, improving techniques, and analyzing existing algorithms (see, e.g., [37]). It also exposes weaknesses in previous abstractions: tournament-based oracles can severely underestimate the information an adversary obtains. Moreover, we show how information from these and other attacks can be formalized and exploited more exhaustively. Finally, our models provide a basis for developing evaluation methods. However, the models are still abstractions, and adversaries can learn more information from actual traces/better models.

Our work also poses several new questions: We have not yet explored strategies that look multiple steps ahead to find the optimal queries. Currently, we target one block at a time – potentially, the number of traces can be reduced by maximizing entropy reduction and using lattice reduction (e.g., [22,6]). In addition, regarding query selection, an adversary could use queries that do not only split along the subkey’s axes. Furthermore, performance improvements through better implementations and algorithmic improvements could increase the number of positions targeted before the recovery becomes computationally infeasible. Moreover, the BP “with restart” is needlessly inefficient for high-noise cases. While these are interesting open questions, we believe that the most pressing matter is finding evaluation models for countermeasures against PC-oracle attacks.

Acknowledgments

We would like to thank Qian Guo, Rei Ueno, and Richard Petri for helpful discussions. Julius Hermelink was funded by the German Federal Ministry of Research,

Technology and Space (BMFTR) through the projects 6GEM (16KISK038). Erik Mårtensson and Maggie Tran were funded by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation. We used the free version of LanguageTool¹²/ltex¹³ for spell/grammar checks (no generative features). For Figures 4 and 5, we used the free version of ChatGPT¹⁴ to find syntax errors; conception and initial figure were done manually. The color schemes for all figures were also suggested by ChatGPT.

References

1. Aguilar-Melchor, C., Aragon, N., Bettaieb, S., Bidoux, L., Blazy, O., Deneuville, J.C., Gaborit, P., Persichetti, E., Zémor, G., Bos, J., Dion, A., Lacan, J., Robert, J.M., Veron, P.: HQC. Tech. rep., National Institute of Standards and Technology (2022), available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/round-4-submissions>
2. Belson, D.: Cloudflare 2024 year in review (2024), <https://blog.cloudflare.com/radar-2024-year-in-review/#13-0-of-tls-1-3-traffic-is-using-post-quantum-encryption>
3. Bos, J.W., Gourjon, M., Renes, J., Schneider, T., van Vredendaal, C.: Masking Kyber: First- and higher-order implementations. IACR TCHES **2021**(4), 173–214 (2021). <https://doi.org/10.46586/tches.v2021.i4.173-214>, <https://tches.iacr.org/index.php/TCHES/article/view/9064>
4. Chari, S., Jutla, C.S., Rao, J.R., Rohatgi, P.: Towards sound approaches to counter-act power-analysis attacks. In: Wiener, M.J. (ed.) CRYPTO’99. LNCS, vol. 1666, pp. 398–412. Springer, Berlin, Heidelberg (Aug 1999). https://doi.org/10.1007/3-540-48405-1_26
5. Chari, S., Rao, J.R., Rohatgi, P.: Template attacks. In: Kaliski, Jr., B.S., Koç, Çetin Kaya., Paar, C. (eds.) CHES 2002. LNCS, vol. 2523, pp. 13–28. Springer, Berlin, Heidelberg (Aug 2003). https://doi.org/10.1007/3-540-36400-5_3
6. Dachman-Soled, D., Ducas, L., Gong, H., Rossi, M.: LWE with side information: Attacks and concrete security estimation. In: Micciancio, D., Ristenpart, T. (eds.) CRYPTO 2020, Part II. LNCS, vol. 12171, pp. 329–358. Springer, Cham (Aug 2020). https://doi.org/10.1007/978-3-030-56880-1_12
7. Dachman-Soled, D., Gong, H., Hanson, T., Kippen, H.: Revisiting security estimation for LWE with hints from a geometric perspective. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023, Part V. LNCS, vol. 14085, pp. 748–781. Springer, Cham (Aug 2023). https://doi.org/10.1007/978-3-031-38554-4_24
8. Dong, H., Guo, Q.: OT-PCA: New key-recovery plaintext-checking oracle based side-channel attacks on HQC with offline templates. Cryptology ePrint Archive, Report 2024/1715 (2024), <https://eprint.iacr.org/2024/1715>
9. Duc, A., Dziembowski, S., Faust, S.: Unifying leakage models: From probing attacks to noisy leakage. In: Nguyen, P.Q., Oswald, E. (eds.) EUROCRYPT 2014. LNCS, vol. 8441, pp. 423–440. Springer, Berlin, Heidelberg (May 2014). https://doi.org/10.1007/978-3-642-55220-5_24

¹² <https://languagetool.org>

¹³ <https://valentjn.github.io/ltex/>

¹⁴ <https://openai.com/index/chatgpt/>

10. Duc, A., Faust, S., Standaert, F.X.: Making masking security proofs concrete - or how to evaluate the security of any leaking device. In: Oswald, E., Fischlin, M. (eds.) EUROCRYPT 2015, Part I. LNCS, vol. 9056, pp. 401–429. Springer, Berlin, Heidelberg (Apr 2015). https://doi.org/10.1007/978-3-662-46800-5_16
11. Engineering, A.S., Architecture: imessage with pq3: The new state of the art in quantum-secure messaging at scale (2024), <https://security.apple.com/blog/imessage-pq3/>
12. Fluhrer, S.: Cryptanalysis of ring-LWE based key exchange with key share reuse. Cryptology ePrint Archive, Report 2016/085 (2016), <https://eprint.iacr.org/2016/085>
13. Fujisaki, E., Okamoto, T.: Secure integration of asymmetric and symmetric encryption schemes. In: Wiener, M.J. (ed.) CRYPTO'99. LNCS, vol. 1666, pp. 537–554. Springer, Berlin, Heidelberg (Aug 1999). https://doi.org/10.1007/3-540-48405-1_34
14. Fujisaki, E., Okamoto, T.: Secure integration of asymmetric and symmetric encryption schemes. Journal of Cryptology **26**(1), 80–101 (Jan 2013). <https://doi.org/10.1007/s00145-011-9114-1>
15. Gallager, R.G.: Low-density parity-check codes. IRE Trans. Inf. Theory **8**(1), 21–28 (1962). <https://doi.org/10.1109/TIT.1962.1057683>, <https://doi.org/10.1109/TIT.1962.1057683>
16. Grosso, V., Standaert, F.X.: ASCA, SASCA and DPA with enumeration: Which one beats the other and when? In: Iwata, T., Cheon, J.H. (eds.) ASIACRYPT 2015, Part II. LNCS, vol. 9453, pp. 291–312. Springer, Berlin, Heidelberg (Nov / Dec 2015). https://doi.org/10.1007/978-3-662-48800-3_12
17. Guo, Q., Johansson, T., Nilsson, A.: A key-recovery timing attack on post-quantum primitives using the Fujisaki-Okamoto transformation and its application on FrodoKEM. In: Micciancio, D., Ristenpart, T. (eds.) CRYPTO 2020, Part II. LNCS, vol. 12171, pp. 359–386. Springer, Cham (Aug 2020). https://doi.org/10.1007/978-3-030-56880-1_13
18. Guo, Q., Mårtensson, E.: Do not bound to a single position: Near-optimal multi-positional mismatch attacks against Kyber and Saber. In: Johansson, T., Smith-Tone, D. (eds.) Post-Quantum Cryptography - 14th International Workshop, PQCrypto 2023. pp. 291–320. Springer, Cham (Aug 2023). https://doi.org/10.1007/978-3-031-40003-2_11
19. Guo, Q., Mårtensson, E., Åström, A.: The perils of limited key reuse: Adaptive and parallel mismatch attacks with post-processing against Kyber. CiC **1**(3), 21 (2024). <https://doi.org/10.62056/a3n5qj888>
20. Hamburg, M., Hermelink, J., Primas, R., Samardjiska, S., Schamberger, T., Streit, S., Strieder, E., van Vredendaal, C.: Chosen ciphertext k-trace attacks on masked CCA2 secure Kyber. IACR TCHES **2021**(4), 88–113 (2021). <https://doi.org/10.46586/tches.v2021.i4.88-113>, <https://tches.iacr.org/index.php/TCHES/article/view/9061>
21. Heinz, D., Kannwischer, M.J., Land, G., Pöppelmann, T., Schwabe, P., Sprenkels, A.: First-order masked Kyber on ARM Cortex-M4. Cryptology ePrint Archive, Report 2022/058 (2022), <https://eprint.iacr.org/2022/058>
22. Hermelink, J., Mårtensson, E., Samardjiska, S., Pessl, P., Rodosek, G.D.: Belief propagation meets lattice reduction: Security estimates for error-tolerant key recovery from decryption errors. IACR TCHES **2023**(4), 287–317 (2023). <https://doi.org/10.46586/tches.v2023.i4.287-317>

23. Hermelink, J., Ning, K.C., Petri, R.: Finding and protecting the weakest link - on side-channel attacks on y in masked ML-DSA. In: Kalai, Y.T., Kamara, S.F. (eds.) CRYPTO 2025, Part V. LNCS, vol. 16004, pp. 3–37. Springer, Cham (Aug 2025). https://doi.org/10.1007/978-3-032-01901-1_1
24. Hermelink, J., Ning, K.C., Petri, R., Strieder, E.: The insecurity of masked comparisons: SCAs on ML-KEM’s FO-transform. In: Luo, B., Liao, X., Xu, J., Kirda, E., Lie, D. (eds.) ACM CCS 2024. pp. 2430–2444. ACM Press (Oct 2024). <https://doi.org/10.1145/3658644.3690339>
25. Hermelink, J., Pessl, P., Pöppelmann, T.: Fault-enabled chosen-ciphertext attacks on Kyber. In: Adhikari, A., Küsters, R., Preneel, B. (eds.) INDOCRYPT 2021. LNCS, vol. 13143, pp. 311–334. Springer, Cham (Dec 2021). https://doi.org/10.1007/978-3-030-92518-5_15
26. Hermelink, J., Streit, S., Mårtensson, E., Petri, R.: A generic framework for side-channel attacks against LWE-based cryptosystems. In: Fehr, S., Fouque, P.A. (eds.) EUROCRYPT 2025, Part VIII. LNCS, vol. 15608, pp. 3–32. Springer, Cham (May 2025). https://doi.org/10.1007/978-3-031-91101-9_1
27. Hesse, D., Feldtkeller, J., Güneysu, T., Hermelink, J., Krausz, M., Land, G.: t -probing (in-)security: Pitfalls on noise assumptions. Cryptology ePrint Archive, Report 2025/1202 (2025), <https://eprint.iacr.org/2025/1202>
28. Huber, M.F., Bailey, T., Durrant-Whyte, H.F., Hanebeck, U.D.: On entropy approximation for gaussian mixture random vectors. In: IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems, MFI 2008, Seoul, South Korea, August 20–22, 2008. pp. 181–188. IEEE (2008). <https://doi.org/10.1109/MFI.2008.4648062>, <https://doi.org/10.1109/MFI.2008.4648062>
29. Kannwischer, M.J., Pessl, P., Primas, R.: Single-trace attacks on Keccak. IACR TCHES **2020**(3), 243–268 (2020). <https://doi.org/10.13154/tches.v2020.i3.243-268>, <https://tches.iacr.org/index.php/TCHES/article/view/8590>
30. Kannwischer, M.J., Schwabe, P., Stebila, D., Wiggers, T.: Improving software quality in cryptography standardization projects. In: IEEE European Symposium on Security and Privacy, EuroS&P 2022 - Workshops, Genoa, Italy, June 6–10, 2022. pp. 19–30. IEEE Computer Society, Los Alamitos, CA, USA (2022). <https://doi.org/10.1109/EuroSPW55150.2022.00010>, <https://eprint.iacr.org/2022/337>
31. Kret, E.: Quantum resistance and the signal protocol (2023), <https://signal.org/blog/pqxdh/>
32. Langlois, A., Stehlé, D.: Worst-case to average-case reductions for module lattices. DCC **75**(3), 565–599 (2015). <https://doi.org/10.1007/s10623-014-9938-4>
33. Li, J., Cheng, C., Shen, M., Chen, P., Guo, Q., Liu, D., Wu, L., Weng, J.: Grafted trees bear better fruit: An improved multiple-valued plaintext-checking side-channel attack against kyber. In: Design, Automation & Test in Europe Conference, DATE 2025, Lyon, France, March 31 - April 2, 2025. pp. 1–7. IEEE (2025). <https://doi.org/10.23919/DATE64628.2025.10992764>, <https://doi.org/10.23919/DATE64628.2025.10992764>
34. MacKay, D.J.C.: Information Theory, Inference & Learning Algorithms. Cambridge University Press, USA (2002)
35. Masure, L., Méaux, P., Moos, T., Standaert, F.X.: Effective and efficient masking with low noise using small-mersenne-prime ciphers. In: Hazay, C., Stam, M. (eds.) EUROCRYPT 2023, Part IV. LNCS, vol. 14007, pp. 596–627. Springer, Cham (Apr 2023). https://doi.org/10.1007/978-3-031-30634-1_20
36. May, A., Nowakowski, J.: Too many hints - when LLL breaks LWE. In: Guo, J., Steinfeld, R. (eds.) ASIACRYPT 2023, Part IV. LNCS, vol. 14441, pp. 106–137.

- Springer, Singapore (Dec 2023). https://doi.org/10.1007/978-981-99-8730-6_4
37. Nagpal, R., Cassiers, G., Primas, R., Knoll, C., Pernkopf, F., Mangard, S.: On loopy belief propagation for SASCAs: An analysis and empirical study of the inference problem. *CiC* **1**(4), 15 (2024). <https://doi.org/10.62056/ay18ksdja>
 38. National Institute of Standards and Technology: Module-lattice-based digital signature standard. Tech. rep., Department of Commerce, Washington, D.C. (2023), federal Information Processing Standards Publication (FIPS) NIST FIPS 204 ipd. <https://doi.org/10.6028/NIST.FIPS.204.ipd>
 39. National Institute of Standards and Technology: Module-lattice-based key-encapsulation mechanism standard. Tech. rep., Department of Commerce, Washington, D.C. (2023), federal Information Processing Standards Publication (FIPS) NIST FIPS 203 ipd. <https://doi.org/10.6028/NIST.FIPS.203.ipd>
 40. National Institute of Standards and Technology: Status report on the fourth round of the nist post-quantum cryptography standardization process. Tech. rep., Department of Commerce, Washington, D.C. (2025), nIST Internal Report NIST IR 8545 <https://doi.org/10.6028/NIST.IR.8545>
 41. Ngo, K., Dubrova, E., Guo, Q., Johansson, T.: A side-channel attack on a masked IND-CCA secure Saber KEM implementation. *IACR TCHES* **2021**(4), 676–707 (2021). <https://doi.org/10.46586/tches.v2021.i4.676-707>, <https://tches.iacr.org/index.php/TCHES/article/view/9079>
 42. Pearl, J.: Reverend bayes on inference engines: A distributed hierarchical approach. In: Waltz, D.L. (ed.) *Proceedings of the National Conference on Artificial Intelligence*, Pittsburgh, PA, USA, August 18–20, 1982. pp. 133–136. AAAI Press (1982), <http://www.aaai.org/Library/AAAI/1982/aaai82-032.php>
 43. Pessl, P., Primas, R.: More practical single-trace attacks on the number theoretic transform. In: Schwabe, P., Thériault, N. (eds.) *LATINCRYPT 2019*. LNCS, vol. 11774, pp. 130–149. Springer, Cham (Oct 2019). https://doi.org/10.1007/978-3-030-30530-7_7
 44. Pessl, P., Prokop, L.: Fault attacks on CCA-secure lattice KEMs. *IACR TCHES* **2021**(2), 37–60 (2021). <https://doi.org/10.46586/tches.v2021.i2.37-60>, <https://tches.iacr.org/index.php/TCHES/article/view/8787>
 45. Primas, R., Pessl, P., Mangard, S.: Single-trace side-channel attacks on masked lattice-based encryption. In: Fischer, W., Homma, N. (eds.) *CHES 2017*. LNCS, vol. 10529, pp. 513–533. Springer, Cham (Sep 2017). https://doi.org/10.1007/978-3-319-66787-4_25
 46. Qin, Y., Cheng, C., Ding, J.: An efficient key mismatch attack on the NIST second round candidate Kyber. *Cryptology ePrint Archive*, Report 2019/1343 (2019), <https://eprint.iacr.org/2019/1343>
 47. Qin, Y., Cheng, C., Zhang, X., Pan, Y., Hu, L., Ding, J.: A systematic approach and analysis of key mismatch attacks on lattice-based NIST candidate KEMs. In: Tibouchi, M., Wang, H. (eds.) *ASIACRYPT 2021, Part IV*. LNCS, vol. 13093, pp. 92–121. Springer, Cham (Dec 2021). https://doi.org/10.1007/978-3-030-92068-5_4
 48. Rajendran, G., Ravi, P., D’Anvers, J.P., Bhasin, S., Chattopadhyay, A.: Pushing the limits of generic side-channel attacks on LWE-based KEMs - parallel PC oracle attacks on Kyber KEM and beyond. *IACR TCHES* **2023**(2), 418–446 (2023). <https://doi.org/10.46586/tches.v2023.i2.418-446>
 49. Ravi, P., Chattopadhyay, A., D’Anvers, J.P., Baksı, A.: Side-channel and fault-injection attacks over lattice-based post-quantum schemes (kyber, dilithium): Survey

- and new results. *ACM Trans. Embed. Comput. Syst.* **23**(2) (Mar 2024). <https://doi.org/10.1145/3603170>, <https://doi.org/10.1145/3603170>
50. Ravi, P., Roy, S.S., Chattopadhyay, A., Bhasin, S.: Generic side-channel attacks on CCA-secure lattice-based PKE and KEMs. *IACR TCHES* **2020**(3), 307–335 (2020). <https://doi.org/10.13154/tches.v2020.i3.307-335>, <https://tches.iacr.org/index.php/TCHES/article/view/8592>
 51. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. In: Gabow, H.N., Fagin, R. (eds.) 37th ACM STOC. pp. 84–93. ACM Press (May 2005). <https://doi.org/10.1145/1060590.1060603>
 52. Shao, M., Liu, Y., Zhou, Y.: Pairwise and parallel: Enhancing the key mismatch attacks on Kyber and beyond. In: Zhou, J., Quek, T.Q.S., Gao, D., Cárdenas, A.A. (eds.) ASIACCS 24. ACM Press (Jul 2024). <https://doi.org/10.1145/3634737.3637661>
 53. Shen, M., Cheng, C., Zhang, X., Guo, Q., Jiang, T.: Find the bad apples: An efficient method for perfect key recovery under imperfect SCA oracles - A case study of Kyber. *IACR TCHES* **2023**(1), 89–112 (2023). <https://doi.org/10.46586/tches.v2023.i1.89-112>
 54. Strieder, E., Ilg, M., Heyszl, J., Unterstein, F., Streit, S.: ASCA vs. SASCA - A closer look at the AES key schedule. In: Kavun, E.B., Pehl, M. (eds.) COSADE 2023. LNCS, vol. 13979, pp. 65–85. Springer, Cham (Apr 2023). https://doi.org/10.1007/978-3-031-29497-6_4
 55. Tanaka, Y., Ueno, R., Xagawa, K., Ito, A., Takahashi, J., Homma, N.: Multiple-valued plaintext-checking side-channel attacks on post-quantum KEMs. *IACR TCHES* **2023**(3), 473–503 (2023). <https://doi.org/10.46586/tches.v2023.i3.473-503>
 56. Ueno, R., Xagawa, K., Tanaka, Y., Ito, A., Takahashi, J., Homma, N.: Curse of re-encryption: A generic power/EM analysis on post-quantum KEMs. *IACR TCHES* **2022**(1), 296–322 (2022). <https://doi.org/10.46586/tches.v2022.i1.296-322>
 57. Veyrat-Charvillat, N., Gérard, B., Standaert, F.X.: Soft analytical side-channel attacks. In: Sarkar, P., Iwata, T. (eds.) ASIACRYPT 2014, Part I. LNCS, vol. 8873, pp. 282–296. Springer, Berlin, Heidelberg (Dec 2014). https://doi.org/10.1007/978-3-662-45611-8_15

Appendix

A Proofs of Lemmas 1 and 2

Lemma 1. *For a sub-key $\mathbf{s}$ and queries $(\mathcal{D}_j = \mathcal{O}(\mathbf{a}_j))_{j \in [K]}$ for that sub-key, we have*

$$P(s_i = v \mid (\mathcal{D}_j = \mathcal{O}(\mathbf{a}_j))_{j \in [K]}) \propto P(s_i = v) \sum_{\mathbf{m} \in [2^N]^K} \prod_{j=0}^{K-1} P_{\mathcal{D}_j}(m_j) \delta_{v \leq a_{j,i}}(m_{j,i}), \quad (29)$$

where $\delta_A(b)$, for a statement A and a bit b , is 1 if and only if A is true and $b = 1$ or A is false and $b = 0$.

Proof. Given the actual message, the queries give no further information. Therefore, we have

$$P(s_i = v \mid (\mathcal{D}_j = \mathcal{O}(\mathbf{a}_j))_{j \in [K]}, m_{\text{actual},j} = m_j \forall j \in [K]) \quad (30)$$

$$= P(s_i = v \mid m_{\text{actual},j} = m_j \forall j \in [K]), \quad (31)$$

and as the messages correspond to queries that represent inequalities, this is, by Bayes' rule, proportional to

$$P(s_i = v \mid m_{\text{actual},j} = m_j \forall j \in [K]) \quad (32)$$

$$\propto P(s_i = v) P(m_{\text{actual},j} = m_j \forall j \in [K] \mid s_i = v) \quad (33)$$

$$\propto P(s_i = v) \prod_{j=0}^{K-1} \delta_{v \leq a_{j,i}}(m_{j,i}). \quad (34)$$

Note that the prior, a binomial distribution in ML-KEM, appears here as not all values fulfilling the inequality are equally likely; instead, they are proportional to the prior and the normalization factor is the sum of the prior probabilities of all values fulfilling the inequality. Now, putting it all together and using the law of total probability, we get

$$P(s_i = v \mid (\mathcal{D}_j = \mathcal{O}(\mathbf{a}_j))_{j \in [K]}) \quad (35)$$

$$= \sum_{\mathbf{m} \in [2^N]^K} P(s_i = v \mid m_{\text{actual},j} = m_j \forall j \in [K]) P_{\mathcal{D}}(\mathbf{m}) \quad (36)$$

$$= \sum_{\mathbf{m} \in [2^N]^K} P(s_i = v \mid m_{\text{actual},j} = m_j \forall j \in [K]) \prod_{j=0}^{K-1} P_{\mathcal{D}_j}(m_j) \quad (37)$$

$$\propto \sum_{\mathbf{m} \in [2^N]^K} P(s_i = v) \prod_{j=0}^{K-1} \delta_{v \leq a_{j,i}}(m_{j,i}) \prod_{j=0}^{K-1} P_{\mathcal{D}_j}(m_j). \quad (38)$$

□

Lemma 2. For K , $(\mathcal{D}_j = \mathcal{O}(\mathbf{a}_j))_{j \in [K]}$, the marginals of p , computed according to (16), equal the conditional probabilities $P(s_i = v \mid (\mathcal{D}_j = \mathcal{O}(\mathbf{a}_j))_{j \in [K]})$.

Proof. We can write the probabilities computed according to (16) as

$$p(\mathbf{v}) = p_{\text{prior}}(\mathbf{v}) \sum_{(m_j)_{j \in [2^N]^K}} \prod_{j=0}^{K-1} P_{\mathcal{D}_j}(m_j) \prod_{i=0}^{N-1} \delta_{v_i \leq a_{j,i}}(m_{j,i}). \quad (39)$$

Computing the marginals for position i' results in

$$\sum_{\mathbf{v}, v_{i'}=v} p_{\text{prior}}(\mathbf{v}) \sum_{(m_j)_{j \in [2^N]^K}} \prod_{j=0}^{K-1} P_{\mathcal{D}_j}(m_j) \prod_{i=0}^{N-1} \delta_{v_i \leq a_{j,i}}(m_{j,i}) \quad (40)$$

$$= \sum_{(m_j)_{j \in [2^N]^K}} \prod_{j=0}^{K-1} P_{\mathcal{D}_j}(m_j) \sum_{\mathbf{v}, v_{i'}=v} p_{\text{prior}}(\mathbf{v}) \prod_{i=0}^{N-1} \delta_{v_i \leq a_{j,i}}(m_{j,i}). \quad (41)$$

Now the last term, $\sum_{\mathbf{v}, v_{i'}=v} p_{\text{prior}}(\mathbf{v}) \prod_{i=0}^{N-1} \delta_{v_i \leq a_{j,i}}(m_{j,i})$, is the sum over all probabilities according to the prior for values that fulfill the inequalities given by the query responses and have $v_{i'} = v$. Given that the prior of $\mathbf{v}$ is just the product over the prior probabilities of the coefficients of $\mathbf{v}$, varying v changes the term by a factor given by the prior probability for the respective v . For the set of $v_{i'} = v$ that fulfills the inequality for i' , this is proportional (in the choice for $v_{i'}$) to the prior; otherwise, it is zero. Therefore, (40) is proportional to the term (11) from Lemma 1. $\square$